

Week 8 Lecture

Welcome to the Week 7 Lecture. In today's lecture, we will learn:

- for..of loops.
- Manipulating arrays with push and pop.
- Object Oriented Programming (OOP).
- How to use classes.
- Inheritance.

1. For ... of loops

The **for...of** loop offers a more straightforward way to iterate over arrays in JavaScript. Unlike the traditional **for** loop, which uses an index, the **for...of** loop directly accesses the array's values. But the **for...of** will always iterate through the array forwards and one element at a time, where the **for** loop is more flexible.

Let's have a look at an example for...of loop:

```
let allFlavours = ['chocolate', 'vanilla', 'spiced', 'honey'];  
  
for (let flavour of allFlavours) {  
  console.log(flavour);  
}
```

In this example, the **for...of** loop goes through each flavour in the **allFlavours** array. During each iteration, the **flavour** variable takes on the value of the current array item. The **console.log(flavour);** statement then prints each flavour to the console. The loop starts with the first item ('chocolate') and proceeds sequentially to the last.

In summary, the **for...of** loop provides an easy and readable way to iterate over array values without managing an index or an iterator.

2. Manipulating arrays

In week 4, we looked at arrays, and so far, we have only used a few arrays that were already filled with values. Arrays can be manipulated in many ways, so let's examine how we can add and remove elements to an array dynamically.

push()

The `push` method allows you to add one or more elements to the end of an array. It then returns the new length of the array. Let's see how this works:

```
let fruits = [7, 4, 6];
fruits.push(11);
console.log(fruits); // Outputs: [7, 4, 6, 11]
```

We can use `for` loops to add multiple elements to an array. This is especially useful when populating an array based on repetitive or patterned data. Here is an example:

```
let numbers = [];
for(let i = 1; i <= 5; i++) {
  numbers.push(i);
}
console.log(numbers); // Outputs: [1, 2, 3, 4, 5]
```

`pop()`

The `pop` method removes the last element from an array and returns that element. This method changes the length of the array. Let's have a look at how that works:

```
let fruits = ['apple', 'banana', 'cherry'];
let lastFruit = fruits.pop();
console.log(fruits); // Outputs: ['apple', 'banana']
console.log(lastFruit); // Outputs: 'cherry'
```

These new ways of manipulating arrays will be helpful later in the lecture.

3. Introduction to OOP

Object-Oriented Programming (OOP) is a programming paradigm based on the concept of "objects." Objects can contain data, in the form of fields (often known as attributes or properties), and code, in the form of procedures (often known as methods).

Let's use an analogy to think about this. Imagine you make a single gingerbread person biscuit from scratch. You would prepare the mixture and then carefully cut out the shape of the biscuit with a knife. That's great for one or two biscuits, but when you want to make a lot of cookies at once, you would use a cookie cutter instead. This would be much faster and make all the cookies the same. A similar thing often happens when we are programming: we might write code for one specific task and then realise we need to replicate that task multiple times. So, how do we create a "cookie cutter" for code?

In Object-Oriented Programming, think of the cookie cutter as a 'class' and each gingerbread person biscuit as an 'object' or 'instance' of that class. Using a class (akin to a cookie cutter) ensures that all your objects (or cookies) maintain consistency. Most importantly, if you ever want to change the properties of the objects (like the shape of the cookies), you modify the class (or

change the cookie cutter) **once**, allowing you to produce as many objects (cookies) as needed with the updated properties.

Key Concepts:

- **Class:** A blueprint for creating objects. It defines the attributes and methods the object will have (the cookie cutter).
- **Object:** An instance of a class (a cookie).
- **Method:** A function defined within a class. Imagine it as an action or behaviour the object can perform, like moving or changing size or colour or calculating something.
- **Attribute:** A variable defined within a class. These are the characteristics of the object, like its colour, size, or the type of icing used.

Why OOP:

- **Reusability:** Once you have a class (cookie cutter), you can use it repeatedly to make many objects (cookies). Classes can be reused in different parts or even in other programs.
- **Scalability:** With OOP, scaling and adding new functionalities is easier. If you wanted to make different types of cookies (e.g., star-shaped, heart-shaped), you'd create new cookie cutters (classes) without changing the baking process.
- **Maintainability:** Just as it's easier to adjust the design of a cookie cutter than to reshape every cookie individually, OOP makes it easier to maintain and debug code. If you want to change how an object derived from a class looks on screen, you can modify the class, and all objects created from it would reflect the change.
- **Modularity:** If you are making something complex, it can be made of reusable parts. Think of this as having different pieces that make up a cookie cutter, like leg and head parts – you could mix and match to make precisely the cookie you need. This makes things faster and more organised. You can do the same with code. Classes can contain other classes, and so on. You can re-use, substitute and adjust functionality quickly and easily (well, at least quicker and easier than if you didn't use classes at all).

OOP and classes have many advanced functions and abilities we won't cover in this course. In the next part of the lecture, we will examine a practical scenario that shows why and how using a class will make your code easier to write and manage.

4. How to use classes

In creative and design contexts, we often start building something and discover new opportunities along the way. This might lead us to change our code as we are experimenting. Here's a scenario that is not the best or most systematic approach to building something, but demonstrates the power of OOP.

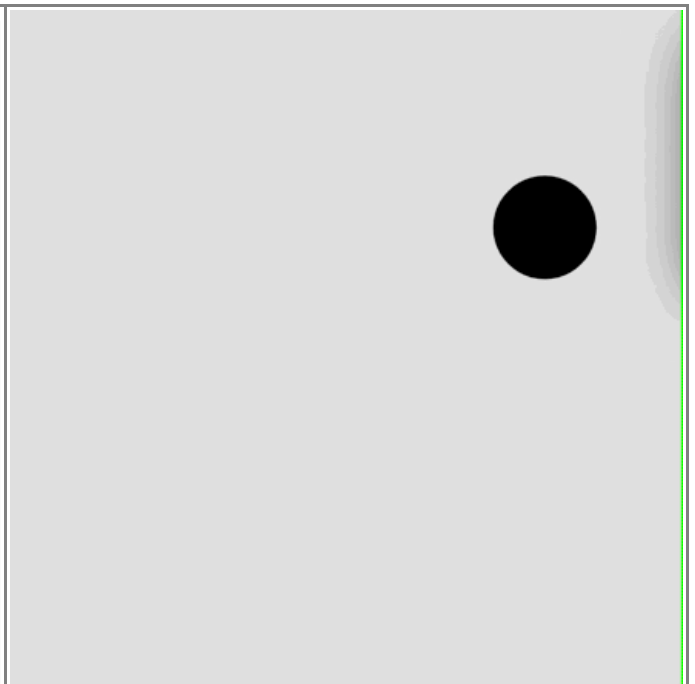
We'll begin with code similar to the week 5 tutorial (the DVD screensaver code). This code creates a circle that starts at a random position and moves diagonally across the screen. When the ball's edge reaches the screen's edge, the ball's movement direction reverses. Here's the basic code for one ball:

```
//lets make some global variables to store the position
let xPos, yPos;
//and the radius of the circle
let radius = 30;
//and the speed of the circle
let xSpeed, ySpeed;

function setup() {
  createCanvas(400, 400);
  //lets set the starting position to a random spot on in the middle section of the canvas
  xPos = random(width/4, 3 * width/4);
  yPos = random(height/4, 3 * height/4);
  //the radius is 30
  radius = 30;
  //the speed is 4 pixels per frame
  xSpeed = 4;
  ySpeed = 4;
}

function draw() {
  //grey background
  background(222);

  //We update the position each frame by adding the speed to the position
  xPos += xSpeed;
  yPos += ySpeed;
  //if the circle hits the left or right edge of the canvas,
  //we reverse the x speed
  if (xPos - radius < 0 || xPos + radius > width) {
    xSpeed *= -1;
  }
  //if the circle hits the top or bottom edge of the canvas,
  //we reverse the y speed
  if (yPos - radius < 0 || yPos + radius > height) {
    ySpeed *= -1;
  }
  //draw a black circle at the current position
  fill(0);
  circle(xPos, yPos, radius * 2);
}
```



This works perfectly. But what if we want two balls?

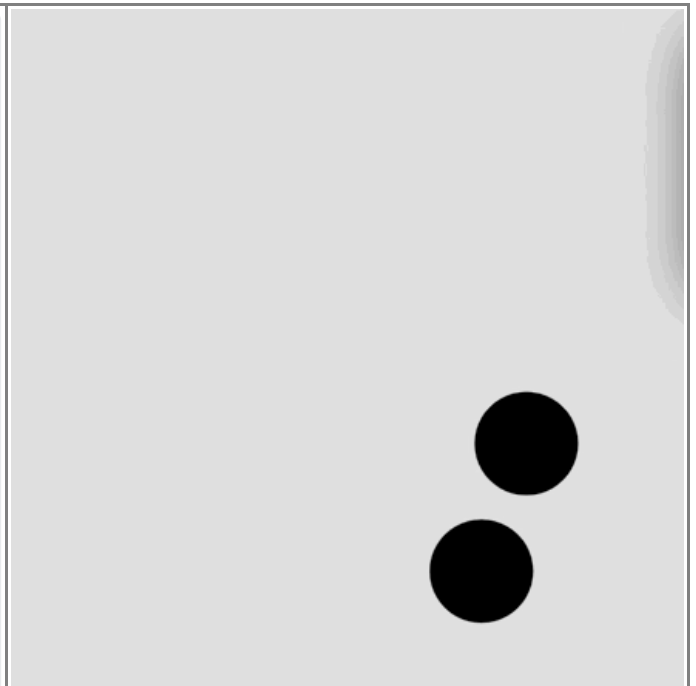
A straightforward approach is to duplicate everything:

```
//lets make some global variables to store the position
let xPos1, yPos1, xPos2, yPos2;
//and the radius of the circle
let radius1 = 30;
//and the speed of the circle
let xSpeed1, ySpeed1, xSpeed2, ySpeed2;

function setup() {
  createCanvas(400, 400);
  //lets set the starting position to a random spot on in the middle section of the canvas
  xPos1 = random(width/4, 3 * width/4);
  yPos1 = random(height/4, 3 * height/4);
  xPos2 = random(width/4, 3 * width/4);
  yPos2 = random(height/4, 3 * height/4);
  //the radius is 30
  radius1 = 30;
  radius2 = 30;
  //the speed is 4 pixels per frame
  xSpeed1 = 4;
  ySpeed1 = 4;
  xSpeed2 = 4;
  ySpeed2 = 4;
}

function draw() {
  //grey background
  background(222);

  //We update the position each frame by adding the speed to the position
  xPos1 += xSpeed1;
  yPos1 += ySpeed1;
  xPos2 += xSpeed2;
  yPos2 += ySpeed2;
  //if the circle hits the left or right edge of the canvas, we reverse the x speed
  if (xPos1 - radius1 < 0 || xPos1 + radius1 > width) {
    xSpeed1 *= -1;
  }
  if (xPos2 - radius2 < 0 || xPos2 + radius2 > width) {
    xSpeed2 *= -1;
  }
  //if the circle hits the top or bottom edge of the canvas, we reverse the y speed
  if (yPos1 - radius1 < 0 || yPos1 + radius1 > height) {
    ySpeed1 *= -1;
  }
  if (yPos2 - radius2 < 0 || yPos2 + radius2 > height) {
    ySpeed2 *= -1;
  }
  //draw a black circle at the current position
  fill(0);
  circle(xPos1, yPos1, radius1 * 2);
  circle(xPos2, yPos2, radius2 * 2);
}
```



And it works!!! Excellent, except it's already very messy and easy to make a mistake if we accidentally swapped any of the ball1 properties with ball2. Let's take a look at how easy it is to make a typo and what happens.

Let's say we want to make 20 balls, this will get out of hand if we keep doing it like this, but we know we can use arrays.

Let's have a look at a version that uses arrays to manage 20 balls:

```
//make an array of xPositions and yPositions
(global variables)
let xPositions = [];
let yPositions = [];

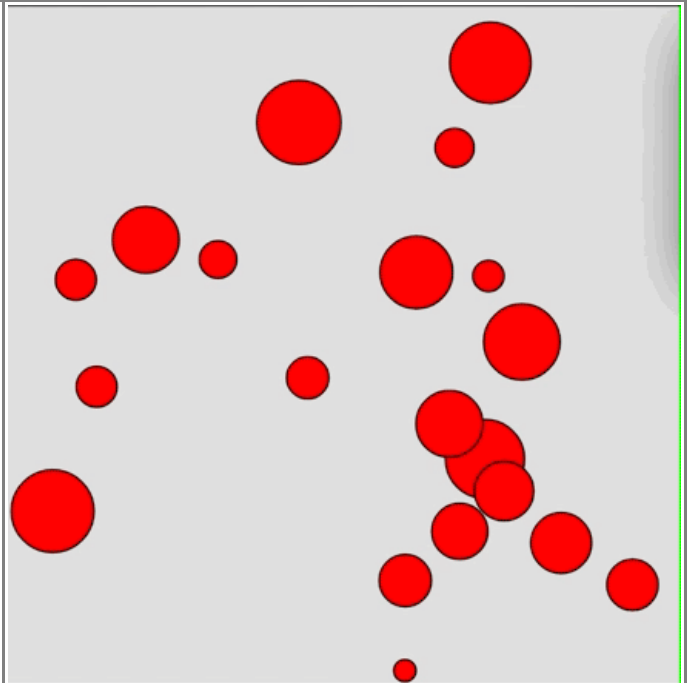
//make an array of xSpeeds and ySpeeds
let xSpeeds = [];
let ySpeeds = [];

//make an array of radii
let radii = [];

function setup() {
  createCanvas(400, 400);
  //make a for loop that iterates 100 times
  for (let i = 0; i < 20; i++) {
    //push a random xPosition and yPosition
    //to the arrays
    xPositions.push(random(width/4, 3 * width/4));
    yPositions.push(random(height/4, 3 * height/4));
    //push a random xSpeed and ySpeed to the
    //arrays
    xSpeeds.push(random(-5, 5));
    ySpeeds.push(random(-5, 5));
    //push a random radius to the array
    radii.push(random(10, 50));
  }
}

function draw() {
  background(222);

  //make a for loop that iterates 20 times
  for (let i = 0; i < 20; i++) {
    //update the position of the circle from
    //the speed
    xPositions[i] += xSpeeds[i];
    yPositions[i] += ySpeeds[i];
    //check if the circle is off the screen
    if (
      xPositions[i] + xSpeeds[i] + radii[i]
/ 2 > width ||
      xPositions[i] + xSpeeds[i] - radii[i]
/ 2 < 0
    ) {
      //if it is, reverse the xSpeed
      xSpeeds[i] *= -1;
    }
    if (
      yPositions[i] + radii[i] / 2 > height
||
      yPositions[i] - radii[i] / 2 < 0
    ) {
      //if it is, reverse the ySpeed
      ySpeeds[i] *= -1;
    }
    //draw the circle
```



```

    fill(255, 0, 0);
    circle(xPositions[i], yPositions[i], rad
ii[i]);
  }
}

```

Ok that is better, but there is still a lot going on in our code, and not much going on our screen, if we wanted to write more code to manage some user interaction or sound or anything else our draw loop will get full of code and hard to read.

Let's see how we can use a Ball class to make this much easier to read and change if we need to.

```

//lets create an empty array to store our ba
lls in
let balls = [];

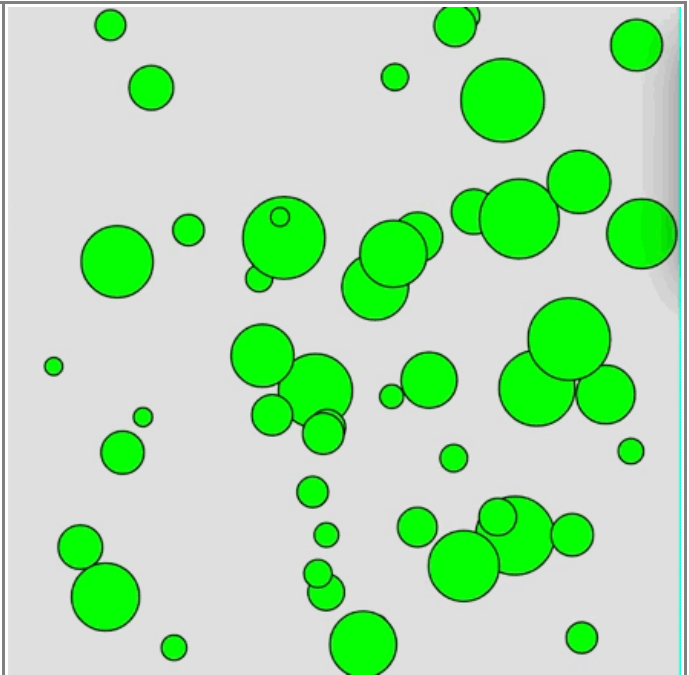
//and a variable for how many balls, lets ma
ke a lot!
const numberOfballs = 50;

function setup() {
  createCanvas(400, 400);
  //here we use a for loop to create a lot of
new balls
  for (let i = 0; i < numberOfballs; i++) {
    //to create a new ball we need to call t
he Ball class constructor and give it values
    //for the starterXPos, starterYPos and s
tarterRadius
    let ball = new Ball(random(width/4, 3*wi
dth/4), random(height/4,3*height/4), random
(10, 50));
    //then we use the push() function to add
the new ball to the balls array
    balls.push(ball);
  }
}

function draw() {
  background(222);
  //this is a for of loop, it is a special t
ype of for loop that is used to loop through
arrays
  //we don't use a numerical iterator like i
n a for loop
  //instead we use a variable to represent e
ach item in the array
  //for each element in the array the ball v
ariable will be set to that element
  //it 'becomes' each element temporarily,
until it has looped through all the elements
  for (let ball of balls) {
    ball.move();
    ball.display();
  }
}

//this is the Ball class, it is a blueprint
for creating balls
//Here we define all the information and fun
ctions that a ball needs
class Ball {
  //this is the class constructor - it is ca
lled when you create a new Ball,
  //it is like the recipe for creating a bal
l, it needs some information to start with
  constructor(starterXPos, starterYPos, star
terRadius) {

```



```

    //the this keyword refers to the object
    that is being created
    //so these parameters are parameters of
    the ball that is being created
    this.ballXPos = starterXPos;
    this.ballYPos = starterYPos;
    this.ballRadius = starterRadius;
    this.ballXSpeed = random(-5, 5);
    this.ballYSpeed = random(-5, 5);
  }

  //we want the ball to move, so we need a m
  ove function
  move() {
    this.ballXPos += this.ballXSpeed;
    this.ballYPos += this.ballYSpeed;

    // Check if the ball is off the screen h
    orizontally
    if (this.ballXPos + this.ballXSpeed + th
    is.ballRadius / 2 > width
    || this.ballXPos + this.ballXSpeed - thi
    s.ballRadius / 2 < 0) {
      this.ballXSpeed *= -1;
    }

    // Check if the ball is off the screen v
    ertically
    if (this.ballYPos + this.ballRadius / 2
    > height
    || this.ballYPos - this.ballRadius / 2 <
    0) {
      this.ballYSpeed *= -1;
    }
  }

  //when we have calculated the new position o
  f the ball we want to display it
  display() {
    fill(0, 255, 0);
    circle(this.ballXPos, this.ballYPos, thi
    s.ballRadius);
  }
}

```

Let's take a closer look at how it works.

When we want to create a class, we need to do it outside of any existing functions, we can do it easily at the bottom of the code (or even in another .js file if we want). We start with the class keyword and then name the class; we use a capital letter at the start of the class name and open and close the class with curly brackets like this:

```

Class Ball{
}

```

Then inside the class we make a constructor, this is a function that is a template for building a new instance of the class, in this case a ball. It sets the class variables to any required values, these can happen internally like the x and y speeds which are set to random values in this case, or like the starting position and radius which are set from parameters in the constructor function.


```

Class Ball{
  constructor(starterXPos, starterYPos, starterRadius) {
    //the this keyword refers to the object that is being created
    //so these parameters are parameters of the ball that is being created
    this.ballXPos = starterXPos;
    this.ballYPos = starterYPos;
    this.ballRadius = starterRadius;
    this.ballXSpeed = random(-5, 5);
    this.ballYSpeed = random(-5, 5);
  }
}

```

We use this keyword inside the constructor function to refer to variables that are members of the Ball class.

To make a new instance of the Ball class we need a variable to hold the instance and then we set that by running the constructor function with the new keyword to make a new instance (like using the cookie cutter).

```

let newBall = new Ball(100, 100, 30);

```

This will make a new ball with the starting x and y positions of 100 and 100 and a radius of 30.

In our new code our ball class also has some functions, move and display. Move contains the same calculations we were performing on all our versions, checking if the ball is past the screen boundary and reversing the direction if so. The class variables are unique to each instance of the ball, so these functions will be calculated for and affect each instance differently.

The draw function is similar and will be used to draw a unique circle for every instance of the ball class.

When we want to access any of the class variables or functions we use dot notation as follows (in this example we are accessing the methods and properties of the first element of the balls array from our example above.

```

balls[0].move();
balls[0].display();
console.log(balls[0]);
console.log(balls[0].ballRadius);
console.log(balls[0].ballXPos);
console.log(balls[0].ballYPos);
console.log(balls[0].ballXSpeed);
console.log(balls[0].ballYSpeed);

```

OOP and classes make our code much better and more efficient, it takes some work and often planning to understand how to use efficient structures. In our tutorial today we will cover a good example of how to use classes in a prototype that will be very useful for your assignments.

4. Inheritance

Inheritance is a programming concept in which we allow objects to inherit properties and methods from other objects. This is super useful when there are a set of methods that we want multiple objects to share. Through the concept of inheritance we can ensure that we don't have to repeat code over and over again for two different objects, since that increases the potential to introduce bugs in our code.

For example, we know that CIRCLES and RECTANGLES are both SHAPES. Thus, they will both possibly need a fillColor() method. Instead of writing individual fillColor() methods for circles and rectangles, we can define a parent class called SHAPE (more on classes soon), and put this method in there. Then, we can declare classes for CIRCLE and RECTANGLE and arrange for them to inherit the methods from SHAPE.

This is exactly like the parent-child relationships we have looked at before for HTML and CSS. The SHAPE is a parent and CIRCLE and RECTANGLES are siblings, children of SHAPE. Spoken another way, a CIRCLE IS-A SHAPE.

That's all for today! Questions?

If you have any questions from the Lecture or Tutorial today - please head to the [Ed discussions page \(https://edstem.org/au/courses/13543/discussion\)](https://edstem.org/au/courses/13543/discussion) and post your question there. When any member of the Teaching Team answers the question - this will be a resource for the whole class to learn from.

Copyright © The University of Sydney. Unless otherwise indicated, 3rd party material has been reproduced and communicated to you by or on behalf of the University of Sydney in accordance with section 113P of the Copyright Act 1968 (Act). The material in this communication may be subject to copyright under the Act. Any further reproduction or communication of this material by you may be the subject of copyright protection under the Act. Do not remove this notice.

Live streamed classes in this unit may be recorded to enable students to review the content. If you have concerns about this, please visit our [student guide \(https://canvas.sydney.edu.au/courses/4901/pages/zoom\)](https://canvas.sydney.edu.au/courses/4901/pages/zoom) and contact the unit coordinator.
[Privacy Statement \(https://www.sydney.edu.au/privacy-statement.html\)](https://www.sydney.edu.au/privacy-statement.html)